

Chapitre 3

Ordonnancement de tâches indépendantes

1. Principaux algorithmes d'ordonnancement de tâches périodiques indépendantes

- A priorités statiques

Rate Monotonic : fondé sur les périodes

Deadline Monotonic : fondé sur les délais critiques

- A priorités dynamiques

Earliest Deadline First (EDF) : fondé sur les échéances

Least Laxity First (LLF) : fondé sur la marge de manœuvre restante

Rate Monotonic (Liu et Layland)

Principe

Priorité = F(Période)

Plus petite période $\Rightarrow$ Plus prioritaire

La priorité est inversement proportionnelle à la période

- RM est **optimal** (pour $P_i = D_i$)

Condition d'ordonnançabilité (pour $P_i = D_i$) :

- CS (théorème de la limite d'utilisation - Lui 73)

$$U(n) = \sum_{i=1}^n \frac{C_i}{P_i} \leq n(2^{\frac{1}{n}} - 1)$$

$$CS(n) = n(2^{1/n} - 1)$$

$$CS(2) = 0,828$$

$$CS(3) = 0,780$$

$$CS(4) = 0,757$$

$$CS(5) = 0,748$$

...

$$CS(n) \rightarrow \log 2 \text{ qd } n \rightarrow \infty$$



- RM **n'est pas optimal** pour $P_i > D_i$

- RM **n'est pas optimal** pour l'ordonnancement non préemptif

RM : exemple 1

$$r_1 = r_2 = 0$$

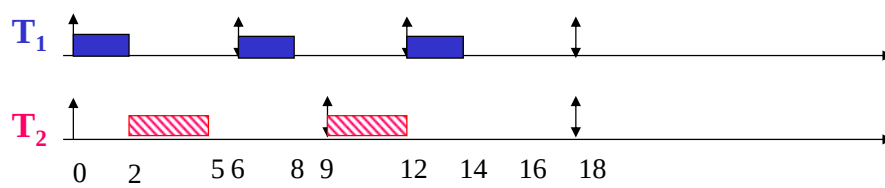
$$D_1 = P_1 = 6$$

$$D_2 = P_2 = 9$$

$$C_1 = 2$$

$$C_2 = 3$$

$$\text{Prio}_2 < \text{Prio}_1$$



$$U = 0,67 < 2(2^{1/2} - 1) = 0,83 \Rightarrow \text{CS vérifiée}$$

RM : exemple 2

$$r_1 = r_2 = r_3 = 0$$

$$D_1 = P_1 = 6$$

$$D_2 = P_2 = 9$$

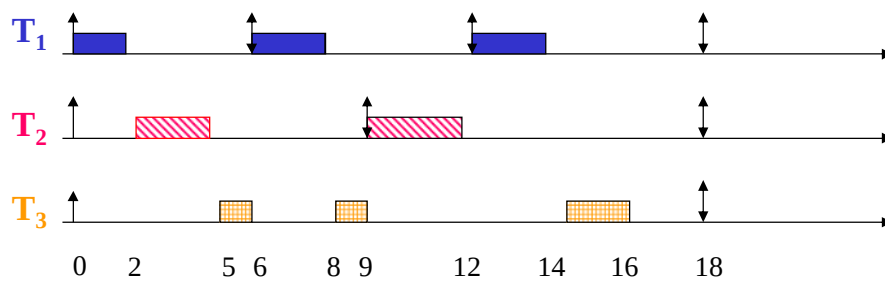
$$D_3 = P_3 = 18$$

$$C_1 = 2$$

$$C_2 = 3$$

$$C_3 = 4$$

$$\text{Prio}_3 < \text{Prio}_2 < \text{Prio}_1$$



$$U = 0,89 > 3(2^{1/3} - 1) = 0,78 \Rightarrow \text{CS non vérifiée}$$

● **Condition nécessaire et suffisante**
Théorème de Lehoczky, Sha et Ding

Un ensemble de tâches (avec $D_i = P_i$) indépendantes est ordonnançable par RM ssi :

$$\forall i, 1 \leq i \leq n, \min_{(k,m) \in R_i} \left\{ \frac{1}{mP_k} \sum_{j=1}^i C_j \left\lceil \frac{mP_k}{P_j} \right\rceil \right\} \leq 1$$

avec :

$$R_i = \left\{ (k,m) \mid 1 \leq k \leq i, m = 1, \dots, \left\lfloor \frac{P_i}{P_k} \right\rfloor \right\}$$

→ Les indices des tâches sont dans l'ordre croissant des périodes : $P_1 \leq P_2 \leq \dots \leq P_n$

Exemple d'utilisation de la CNS du théorème de Lehoczky et al. (1/3)

Exemple de configuration

Tâche	P_i	C_i	r_0
T1	5	2	0
T2	10	2	0
T3	20	4	0



$$\sum_{i=1}^3 \frac{C_i}{P_i} \leq 3(2^{\frac{1}{3}} - 1)$$

$$\sum_{i=1}^3 \frac{C_i}{P_i} = \frac{2}{5} + \frac{2}{10} + \frac{4}{20} = 0.8 > 3(2^{\frac{1}{3}} - 1) = 0.779$$

CS de Liu non satisfaite

$$R_1 = \left\{ (k, m) \mid 1 \leq k \leq 1, m = 1, \dots, \left\lfloor \frac{P_1}{P_k} \right\rfloor \right\} = \{(1, 1)\}$$

$$i = 1, \min_{(k, m) \in R_1} \left\{ \frac{1}{mP_1} \sum_{j=1}^1 C_j \left\lceil \frac{mP_k}{P_j} \right\rceil \right\} \leq 1$$

$$\frac{1}{P_1} C_1 \left\lceil \frac{P_1}{P_1} \right\rceil = \frac{C_1}{P_1} = \frac{2}{5} \leq 1 \Rightarrow \mathbf{T_1 \text{ est ordonnançable}}$$

Exemple d'utilisation de la condition NS du Théorème de Lehoczky et al. (2/3)

$$i = 2, \min_{(k,m) \in R_2} \left\{ \frac{1}{mP_k} \sum_{j=1}^2 C_j \left\lceil \frac{mP_k}{P_j} \right\rceil \right\} \leq 1$$

$$(1,1) \Rightarrow \frac{1}{P_1} \sum_{j=1}^2 C_j \left\lceil \frac{P_1}{P_j} \right\rceil = \frac{C_1}{P_1} \left\lceil \frac{P_1}{P_1} \right\rceil + \frac{C_2}{P_1} \left\lceil \frac{P_1}{P_2} \right\rceil = \frac{C_1}{P_1} + \frac{C_2}{P_1} = \frac{4}{5} \leq 1 \quad \Rightarrow T_2 \text{ est ordonnançable}$$

$$(1,2) \Rightarrow \frac{1}{2P_1} \sum_{j=1}^2 C_j \left\lceil \frac{2P_1}{P_j} \right\rceil = \frac{1}{2P_1} \left(C_1 \left\lceil \frac{2P_1}{P_1} \right\rceil + C_2 \left\lceil \frac{2P_1}{P_2} \right\rceil \right) = \frac{2C_1 + C_2}{2P_1} = \frac{6}{10} \leq 1 \quad \Rightarrow T_2 \text{ est ordonnançable}$$

$$(2,1) \Rightarrow \frac{1}{P_2} \sum_{j=1}^2 C_j \left\lceil \frac{P_2}{P_j} \right\rceil = \frac{1}{P_2} \left(C_1 \left\lceil \frac{P_2}{P_1} \right\rceil + C_2 \left\lceil \frac{P_2}{P_2} \right\rceil \right) = \frac{2C_1 + C_2}{P_2} = \frac{6}{10} \leq 1 \quad \Rightarrow T_2 \text{ est ordonnançable}$$

Il suffit de trouver un couple (k,m) qui vérifie la relation ≤ 1
Ici on a traité les trois couples (k,m), juste pour mieux comprendre l'application du théorème

Exemple d'utilisation de la condition NS du Théorème de Lehoczky et al. (3/3)

$$R_3 = \left\{ (k, m) \mid 1 \leq k \leq 3, m = 1, \dots, \left\lfloor \frac{P_3}{P_k} \right\rfloor \right\} = \{(1,1), (1,2), (1,3), (1,4), (2,1), (2,2), (3,1)\}$$

$$(1,1) \Rightarrow \frac{1}{P_1} \sum_{j=1}^3 C_j \left\lceil \frac{P_1}{P_j} \right\rceil = \frac{C_1}{P_1} \left\lceil \frac{P_1}{P_1} \right\rceil + \frac{C_2}{P_1} \left\lceil \frac{P_1}{P_2} \right\rceil + \frac{C_3}{P_1} \left\lceil \frac{P_1}{P_3} \right\rceil = \frac{C_1}{P_1} + \frac{C_2}{P_1} + \frac{C_3}{P_1} = \frac{8}{5} \geq 1 \quad \Rightarrow \text{On ne peut pas conclure encore}$$

$$(1,2) \Rightarrow \frac{1}{2P_1} \sum_{j=1}^3 C_j \left\lceil \frac{2P_1}{P_j} \right\rceil = \frac{C_1}{2P_1} \left\lceil \frac{2P_1}{P_1} \right\rceil + \frac{C_2}{2P_1} \left\lceil \frac{2P_1}{P_2} \right\rceil + \frac{C_3}{2P_1} \left\lceil \frac{2P_1}{P_3} \right\rceil = \frac{2C_1}{2P_1} + \frac{C_2}{2P_1} + \frac{C_3}{2P_1} = \frac{10}{10} \leq 1 \quad \Rightarrow T_3 \text{ est ordonnançable}$$

On pourrait faire aussi le test pour les 5 couples (k,m) qui restent mais c'est inutile.

Deadline Monotonic (Leung et Whitehead)

Principe

Priorité = F(Délai critique)

Plus petit délai critique $\Rightarrow$ Plus prioritaire

La priorité est inversement proportionnelle au délai critique

- DM est **optimal** pour l'ordonnancement préemptif si $P_i \geq D_i$

Condition d'ordonnançabilité : (Leung et Whitehead)

- CS :
$$\sum_{i=1}^n \frac{C_i}{D_i} \leq n(2^{\frac{1}{n}} - 1)$$



- DM est **optimal** pour le non-préemptif si $\forall i, \forall k, C_i \leq C_k \Rightarrow D_i \leq D_k$
- DM **n'est pas optimal** pour l'ordonnancement non préemptif en général

RM et DM : exemple

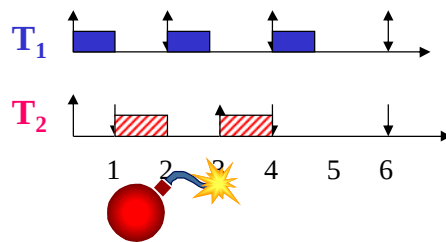
$$r_1 = r_2 = 0$$

$$D_1 = P_1 = 2$$

$$D_2 = 1 \quad P_2 = 3$$

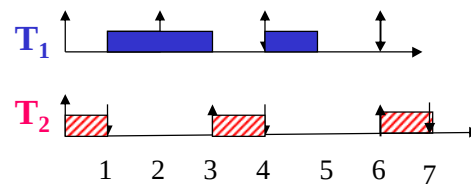
$$C_1 = 1$$

$$C_2 = 1$$



RM

$$\text{Pr}io_2 < \text{Pr}io_1$$



DM

$$\text{Pr}io_1 < \text{Pr}io_2$$

● **Condition nécessaire et suffisante (condition du temps de réponse)**
Théorème de Audsley et al.

Un ensemble de tâches (avec $D_i \leq P_i$) indépendantes et indexées selon leurs échéances (i.e. : $i > j \Rightarrow D_i \geq D_j$) est ordonnançable par DM ssi :

$$\forall i, 1 \leq i \leq n, R_i = C_i + \sum_{j=1}^{i-1} \left\lceil \frac{R_i}{P_j} \right\rceil C_j \leq D_i$$

$$R_1 = C_1$$

R_i ($i \geq 2$) se calcule de manière itérative comme suit :

- On fixe R_i^0 à 0
- On calcule R_i^1 à partir de R_i^0
- ...
- On calcule R_i^m à partir de R_i^{m-1}
- On s'arrête quand R_i^{m+1} est égal à R_i^m

Exemple d'utilisation de la condition NS du Théorème de Audsley

Exemple de configuration

Tâche	P_i	C_i	D_i
T1	12	3	8
T2	20	6	10



$$\sum_{i=1}^2 \frac{C_i}{D_i} \leq 2(2^{\frac{1}{2}} - 1)$$

$$\sum_{i=1}^2 \frac{C_i}{D_i} = \frac{3}{8} + \frac{6}{10} = 0.975 > 2(2^{\frac{1}{2}} - 1) = 0.83$$

CS de Leung et Whitehead non satisfaite

$$i = 1 \Rightarrow R_1 = C_1 = 3 \leq D_1 = 8$$

$$i = 2 \Rightarrow R_2 = C_2 + \sum_{j=1}^1 \left\lceil \frac{R_j}{P_j} \right\rceil C_j = C_2 + C_1 \left\lceil \frac{R_1}{P_1} \right\rceil \leq D_2$$

$$R_2^0 = 0$$

$$R_2^1 = C_2 + C_1 \left\lceil \frac{R_1^0}{P_1} \right\rceil = C_2 = 6$$

$$R_2^2 = C_2 + C_1 \left\lceil \frac{R_1^1}{P_1} \right\rceil = 6 + 3 \left\lceil \frac{6}{12} \right\rceil = 9$$

$$R_2^3 = C_2 + C_1 \left\lceil \frac{R_1^2}{P_1} \right\rceil = 6 + 3 \left\lceil \frac{9}{12} \right\rceil = 9 \leq 10$$



T₁ et T₂ sont ordonnançables

Calcul du temps de réponse maximum de tâche périodique à priorité fixe

Cas préemptif (cas synchrone i.e. $\forall k, r_k = 0$)

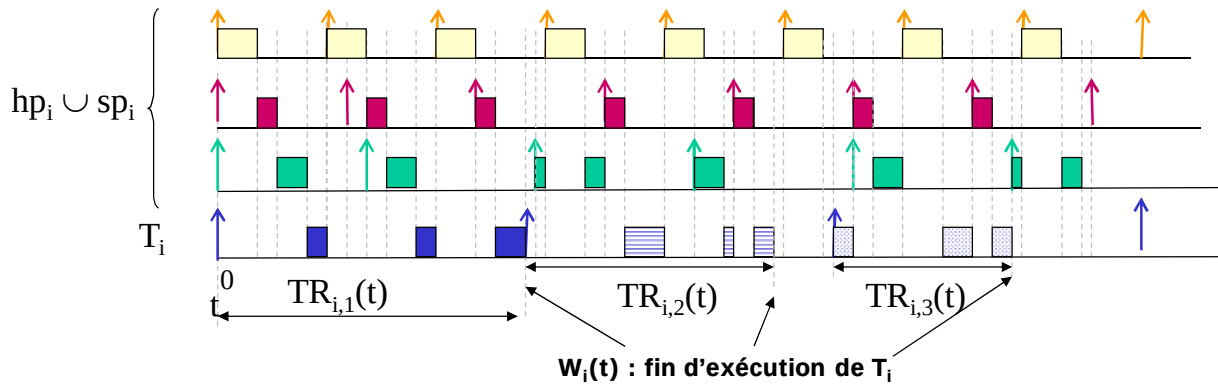
Cas $D_i \leq P_i$

Une tâche plus prioritaire peut retarder T_i jusqu'à la fin de son exécution

hp_i : ensemble de tâches plus prioritaires que la tâche T_i

sp_i : ensemble de tâches de même priorité que la tâche T_i

$Tr_i(t)$: temps de réponse de la tâche T_i devenue activable à t



Le pire cas apparaît pendant la première période d'activité du processeur

$$TR_i = W_i(0) = C_i + \sum_{T_j \in hp_i \cup sp_i} \left\lceil \frac{W_i(0)}{P_j} \right\rceil C_j$$

Earliest Deadline First (Liu et Layland)

Principe

Echéance la plus proche $\Rightarrow$ Plus prioritaire

La priorité est inversement proportionnelle à l'échéance

EDF est **optimal** pour les systèmes de tâches **indépendantes** (pour $P_i = D_i$)

CNS pour les systèmes à échéance sur requête ($P_i = D_i$) :

$$U \leq 1 \quad (\text{Liu et Layland})$$

CS pour les systèmes avec $P_i \neq D_i$) : $\sum_{i=1}^n \frac{C_i}{\min(D_i, P_i)} \leq 1$

EDF : exemple

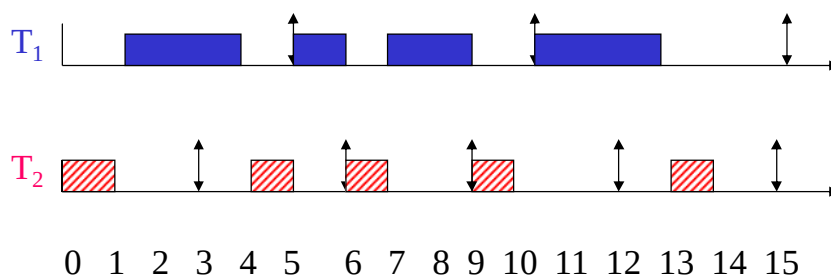
$$r_1 = r_2 = 0$$

$$D_1 = P_1 = 5$$

$$D_2 = P_2 = 3$$

$$C_1 = 3$$

$$C_2 = 1$$



$$U = 3/5 + 1/3 = 14/15 < 1 \Rightarrow \text{le système est ordonnançable par EDF}$$

● **Condition nécessaire et suffisante (condition de la demande de processeur)**
Théorème de Jeffay et Stone

Un ensemble de tâches (avec $D_i \leq P_i$) est ordonnançable par EDF ssi :

$$\forall t > 0, t \geq \sum_{i=1}^n \left(\left\lfloor \frac{t - D_i}{P_i} \right\rfloor + 1 \right) \times C_i$$

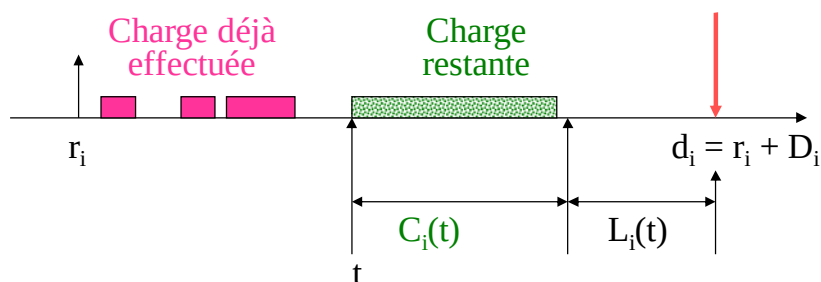
Least Laxity First (Sorenson)

Principe :

$$\text{Laxité}(t) = Li(t) = r_i + D_i - t - C_i(t)$$

Laxité plus petite $\Rightarrow$ Prioritaire plus élevée

La priorité est inversement proportionnelle à la laxité

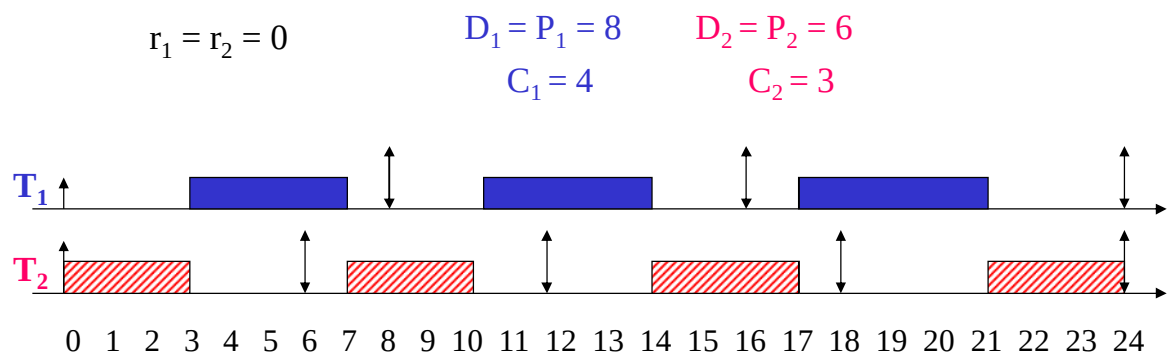


Même capacité d'ordonnancement que EDF

Plus équitable que EDF (utile en cas d'allocation équitable)

Défaut : Nombre élevé de changements de contexte
Recalcul des priorités toutes les unités de temps

EDF et LLF : exemple 1 (1/2)



Avec EDF : 6 changements de contexte

EDF et LLF : exemple 1 (2/2)

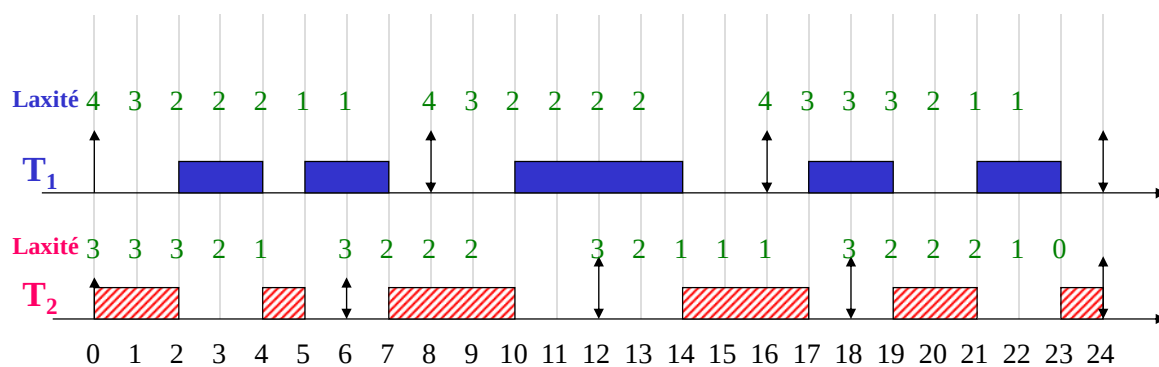
$$r_1 = r_2 = 0$$

$$D_1 = P_1 = 8$$

$$D_2 = P_2 = 6$$

$$C_1 = 4$$

$$C_2 = 3$$

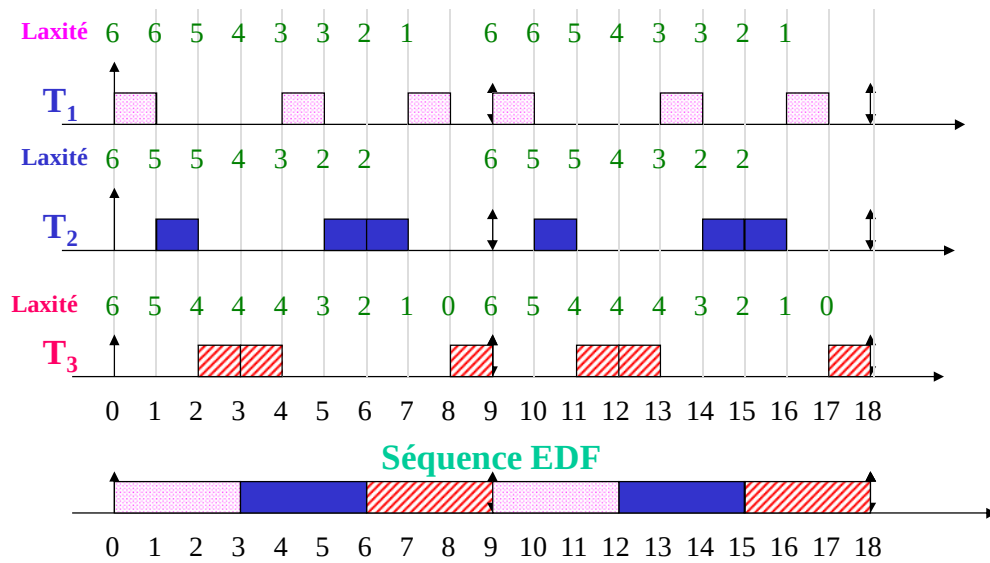


Avec LLF : 10 changements de contexte

EDF et LLF : exemple 2

$$r_1 = 0, D_1 = 9, P_1 = 9, C_1 = 3 \quad r_2 = 0, D_2 = 9, P_2 = 9, C_2 = 3$$

$$r_3 = 0, D_3 = 9, P_3 = 9, C_3 = 3$$



2. Ordonnancement de tâches apériodiques indépendantes

- Cas des tâches apériodiques avec **contraintes de délai**

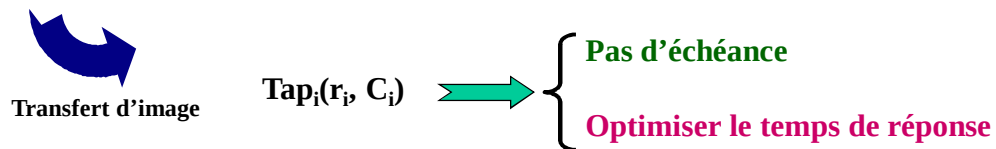


Alarme
coupure courant

$\text{Tap}_i(r_i, C_i, D_i)$

- **Tâches critiques** : transformation (à la **conception**) de tâches apériodiques en tâches périodiques. Dans ce cas l'intervalle minimum doit être connu et sera utilisé comme période. On parle ici de tâche **sporadique**.
Ou en utilisant un serveur
- **Tâches non critiques** : acceptation dans les temps creux d'une séquence rigide de tâches (utilisation de **test d'acceptation**)

- Cas des tâches apériodiques **sans échéance**



- **Traitement en arrière plan** (background processing)

les tâches apériodiques sont traitées pendant les temps d'oisiveté du processeur (quand il n'y a pas de tâches périodiques à exécuter).

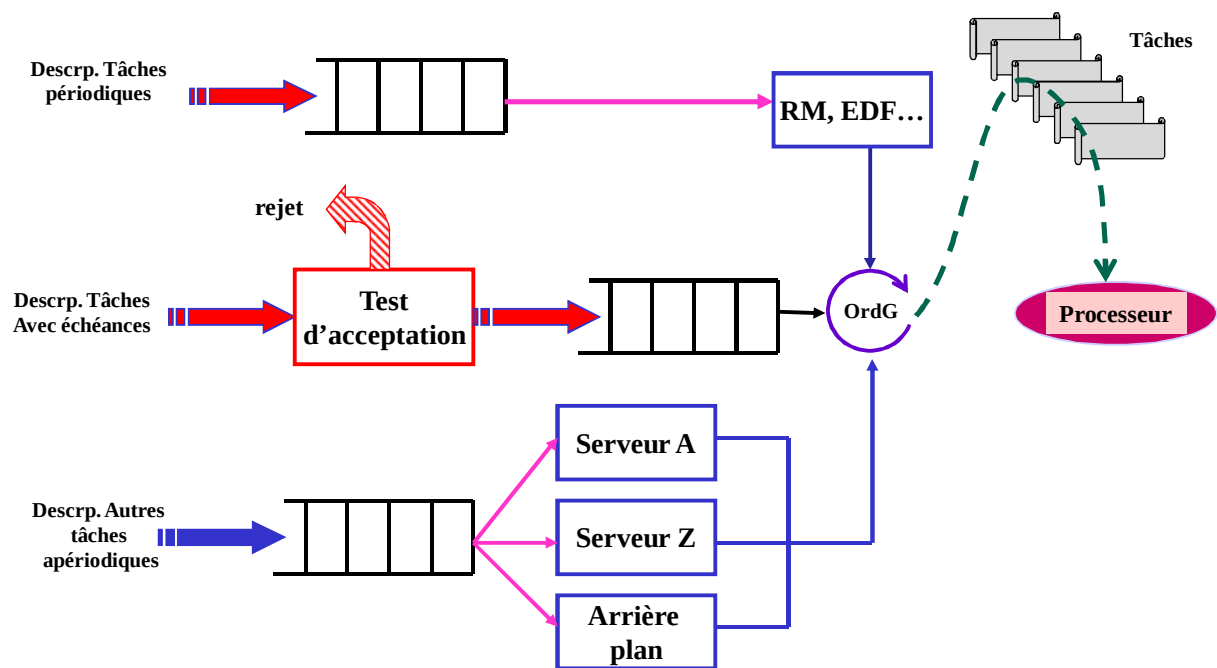
- **Utilisation d'un serveur de tâches apériodiques**

- **Vol de temps creux**

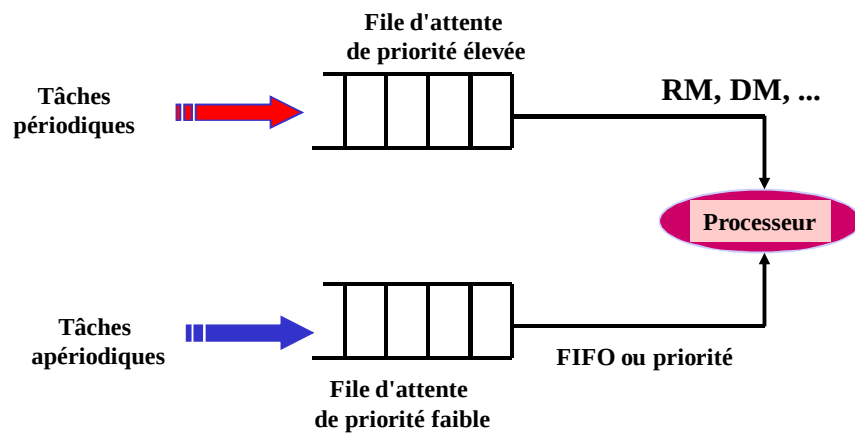
Types de serveurs de tâches aperiodiques

- **serveur à scrutation**
- **serveur ajournable**
- **serveur sporadique**
- **serveur à échange de priorité**
- **serveur Earliest Deadline Last (EDL)**
- **serveur à largeur de bande maximale**
- **autres**

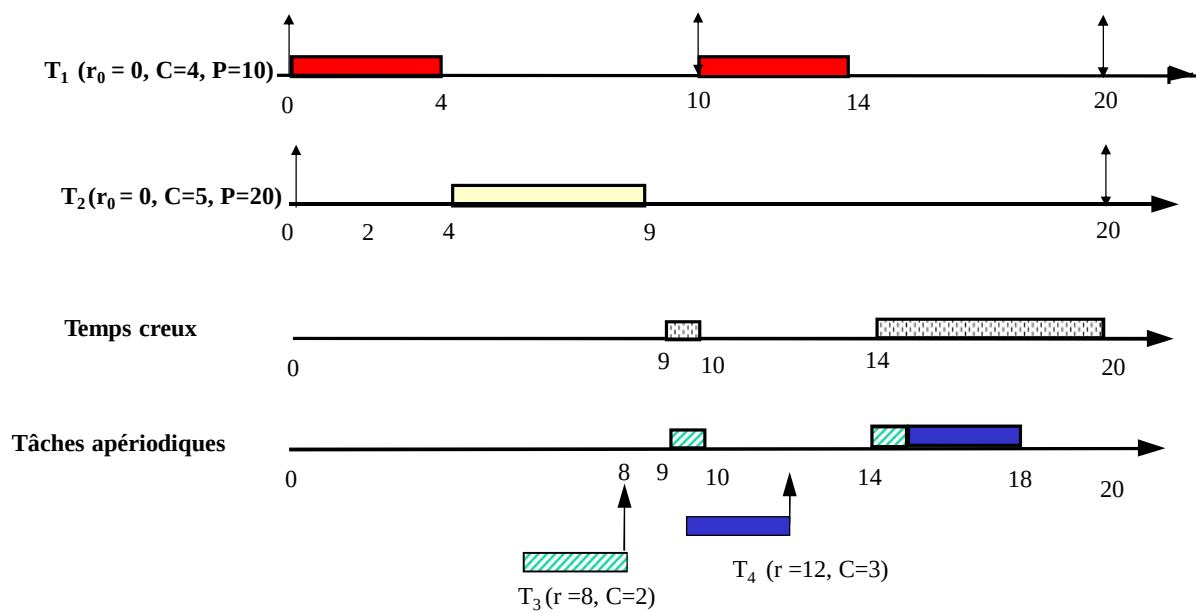
Différentes stratégies pour le traitement de tâches apériodiques



Traitement en arrière plan



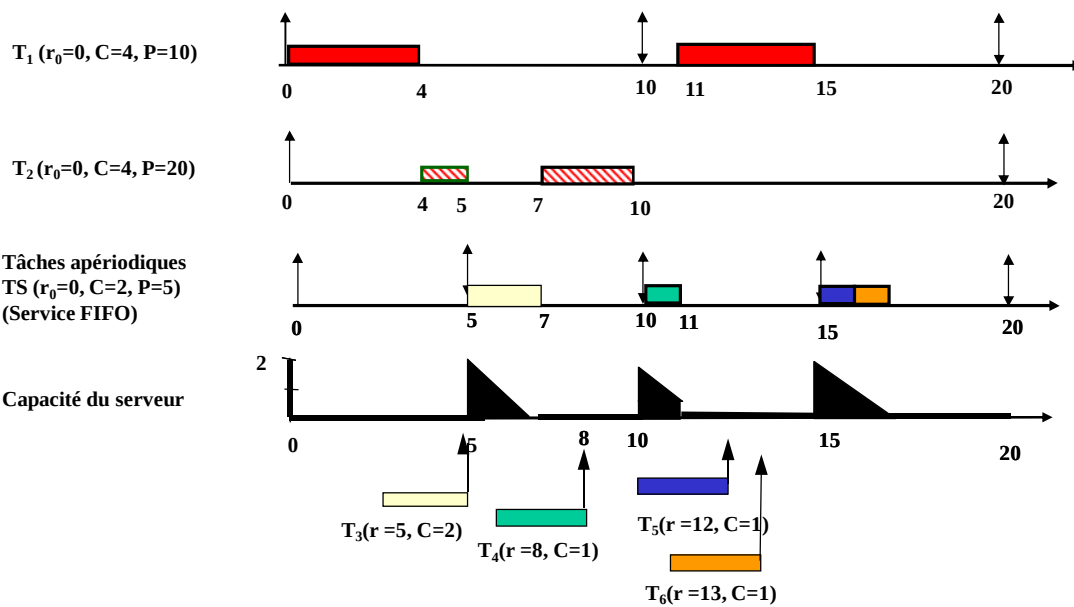
Traitement en arrière plan : exemple



Serveur à scrutation (Polling server) **(Lehoczky, Sha et Strosnider)**

- Une tâche périodique spécifique, appelée serveur, est dédiée aux traitements des requêtes apériodiques.
- Le serveur à scrutation traite les tâches apériodiques qu'il trouve en attente lors de son activation.
- S'il n'y a pas de tâches en attente, le serveur se suspend et attend sa prochaine période d'activation. **Il perd sa capacité de service.**
- Un ou plusieurs serveurs à scrutation peuvent être utilisés selon les besoins.

Serveur à scrutation avec RM : exemple



Test de garantie de tâches apériodiques en cas d'utilisation de Serveur à scrutation

- P_s : période du serveur
- C_s : capacité du serveur

Cas d'une seule tâche apériodique

- Tâche apériodique définie par:
 r_a (instant d'arrivée), C_a (durée d'exécution) et D_a (délai)
- Une tâche apériodique (quand elle est seule) peut attendre P_s au plus avant de s'exécuter
- Si $C_a \leq C_s$ alors garantie si $2P_s \leq D_a$
- Condition suffisante : Si C_a est quelconque alors garantie si $P_s \left(\left\lceil \frac{C_a}{C_s} \right\rceil + 1 \right) \leq D_a$

- **Condition nécessaire et suffisante quand le serveur a la plus haute priorité parmi toutes les tâches périodiques**

$$\left(\left\lfloor \frac{C_a}{C_s} \right\rfloor + \left\lceil \frac{r_a}{P_s} \right\rceil \right) P_s + \left(C_a - \left\lfloor \frac{C_a}{C_s} \right\rfloor C_s \right) \leq r_a + D_a$$

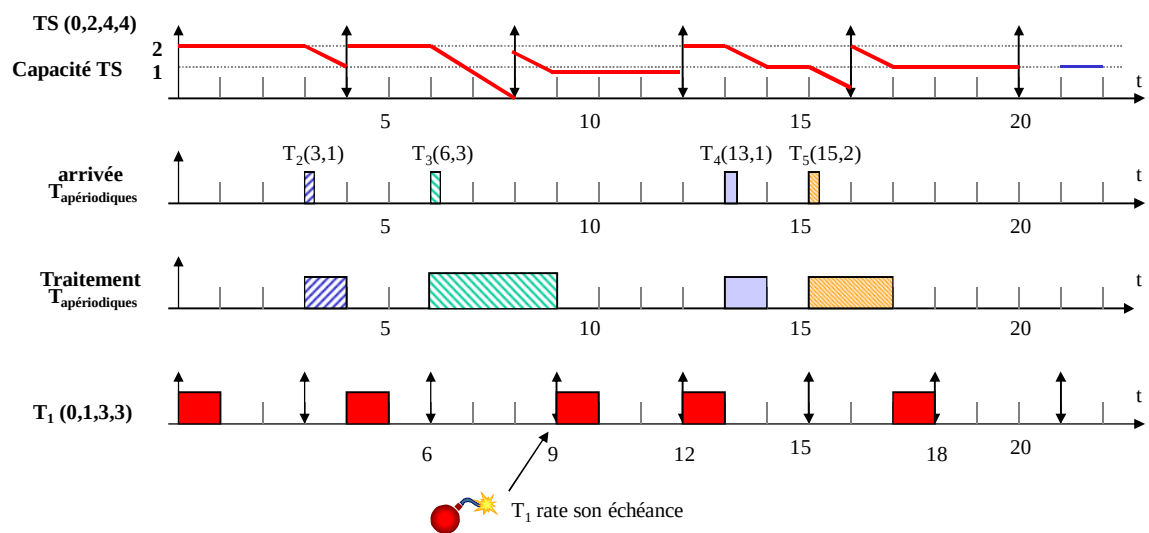
Cas de k tâches apériodiques ordonnées selon leur délai

- **Des conditions (CS et CNS) complexes existent**

Serveur ajournable (deferrable server) (Lehoczky, Sha et Strosnider)

- Une tâche périodique spécifique, appelée serveur, est dédiée aux traitements des requêtes apériodiques.
- Le serveur ajournable traite les tâches apériodiques qu'il trouve en attente lors de son activation.
- S'il n'y a pas de tâches en attente, le serveur se suspend et attend la prochaine période d'activation. **Il conserve sa capacité de service.**
- Si une requête apériodique arrive alors que le serveur ajournable est suspendu, elle est exécutée si la capacité du serveur n'est pas nulle (d'où violation du principe de RM)
- En début de période, la capacité du serveur est remise dans son état initial indépendamment de la capacité précédemment consommée.
- Un ou plusieurs serveurs ajournables peuvent être utilisés selon les besoins.

Serveur ajournable avec RM : exemple 1



Test de garantie de tâches apériodiques en cas d'utilisation de Serveur ajournable

- P_s : période du serveur
- $C_s(t)$: capacité du serveur à l'instant t

Cas d'une seule tâche apériodique et serveur avec la plus haute priorité

- Tâche apériodique définie par r_a (instant d'arrivée), C_a (durée d'exécution) et D_a (délai)
- Une tâche apériodique peut s'exécuter dès qu'elle arrive si $C_s(r_a) > 0$
- Condition nécessaire et suffisante :

$$\left\{ \begin{array}{l} r_a + C_a \leq r_a + D_a \text{ Si } C_a \leq C_s(r_a) \\ \left(\left\lfloor \frac{C_a - \Delta_a}{C_s} \right\rfloor + \left\lceil \frac{r_a}{P_s} \right\rceil \right) P_s + C_a - \Delta_a - \left\lfloor \frac{C_a - \Delta_a}{C_s} \right\rfloor C_s \leq r_a + D_a \text{ Sinon} \end{array} \right.$$

$$\Delta_a = \min \left[C_s(r_a), \left(\left\lceil \frac{r_a}{P_s} \right\rceil P_s - r_a \right) \right]$$

Serveur sporadique (sporadic server)

(Sprunt, Sha et Lehoczky)

- **Le serveur sporadique permet d'améliorer les temps de réponse des tâches apériodiques, sans diminuer le taux d'utilisation du processeur des tâches périodiques.**
- **Le serveur sporadique ne réapprovisionne pas sa capacité de manière périodique comme les serveurs à scrutation et ajournable, mais seulement quand elle a été consommée par des tâches apériodiques.**
- **Efficacité du concept de serveur dynamique**
 - **Utile s'il existe plusieurs serveurs qui fonctionnent en parallèle dans le système. On donne plus de capacité à celui qui en a besoin.**
 - **Encore plus utile si on peut adapter dynamiquement la capacité maximale du SS.**
- **Il existe plusieurs formes de serveurs sporadiques, pour priorités fixes ou dynamiques (les serveurs sporadiques se distinguent par la manière de consommer et réapprovisionner la capacité de service)**

Fonctionnement d'un serveur sporadique simple (avec priorités fixes)

PR_{exe} : priorité de la tâche en cours d'exécution.

PR_s : priorité du serveur sporadique.

P_s : période du serveur sporadique. C_s : capacité maximale du serveur sporadique.

RT : instant où aura lieu le réapprovisionnement de la capacité du SS.

RA : quantité de réapprovisionnement qui sera rajoutée à la capacité du SS.

Actif : le SS est dit **actif** quand $PR_{exe} \geq PR_s$. (SS actif ne signifie pas qu'il consomme du temps CPU)

Inactif : le SS est dit **inactif** quand $PR_{exe} < PR_s$.

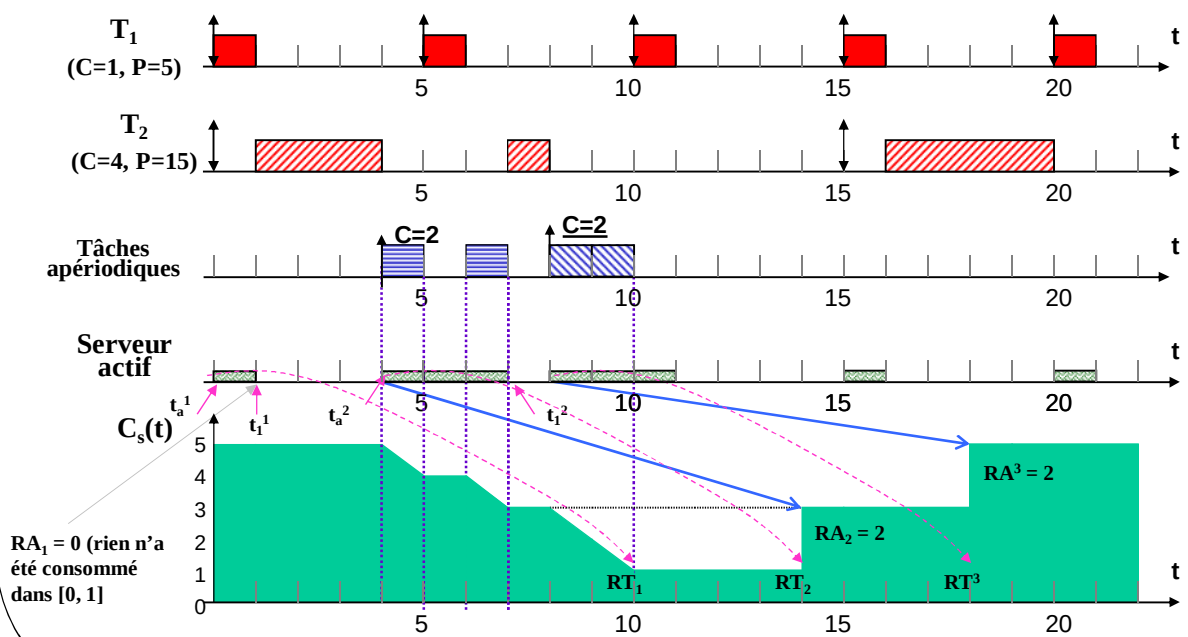
Quand une tâche apériodique arrive, elle s'exécute si la capacité du serveur n'est pas nulle et si la priorité du serveur le permet (les tâches apériodiques s'exécutent avec le même niveau de priorité que le serveur sporadique).

Règle de réapprovisionnement de la capacité du serveur sporadique

- A l'initialisation, la capacité du SS est approvisionnée avec la valeur maximale C_s .
- L'instant de réapprovisionnement RT est fixé dès le moment où le SS devient activable et $C_s > 0$ (soit t_a ce moment). La valeur de RT est fixé à $t_a + P_s$.
- La quantité de réapprovisionnement, RA , à effectuer à l'instant RT est fixée au moment où le SS devient inactif ou quand sa capacité est épuisée (soit t_1 ce moment). La valeur de RA est fixée à une quantité égale à celle consommée durant l'intervalle $[t_a, t_1]$.

Serveur sporadique avec RM : exemple (1/2)

$T_s (r_0 = 0, C_s = 5, P_s = 10, D_s = 10)$



Serveur sporadique avec RM : exemple (2/2)

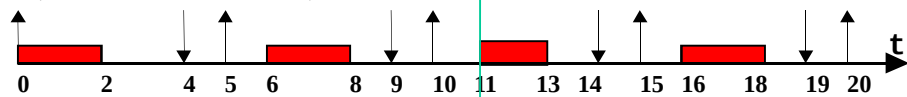
- A $t = 0$, T_1 (tâche la plus prioritaire) est activée et le SS (serveur sporadique) devient actif.
Comme $C_s > 0$, RT_1 est fixé à $t + P_s = 10$. ($t_a^1 = 0$)
- A $t = 1$, T_1 se termine, le SS devient inactif, car il n'a pas de tâche apériodique à servir.
 RA_1 (pour $t_1^1 = 1$) est égale à 0 et aucun réapprovisionnement n'est effectué à l'instant RT_1 , car aucune quantité de la capacité du SS n'a été consommée dans l'intervalle $[0, 1]$.
- A $t = 4$, une tâche apériodique, T_3 , arrive. Comme $C_s > 0$, le SS devient actif et T_3 s'exécute.
Par conséquent un réapprovisionnement est prévu à $RT_2 = t + P_s = 14$. ($t_a^2 = 4$)
- A $t = 5$, la tâche apériodique T_3 est préemptée par T_1 . Le SS reste actif pendant la préemption.
- A $t = 6$, T_1 se termine et T_3 reprend son exécution et se termine à $t = 7$.
- A $t = 7$, le SS devient inactif. La quantité de réapprovisionnement à faire à RT_2 est égale à la capacité consommée pendant l'intervalle $[4, 7]$, c'est-à-dire, $RA_2 = 2$.
- A $t = 8$, le SS devient actif pour servir une nouvelle tâche apériodique, T_4 , arrivée à $t = 8$. Un nouveau réapprovisionnement est prévu à $RT_3 = t + P_s = 18$. ($t_a^3 = 8$)
- A $t = 10$, T_4 se termine, mais le SS reste actif car $PR_{exe} \geq PR_s$.
- A $t = 11$, le SS devient inactif et la quantité de réapprovisionnement à faire à l'instant RT_3 est fixée à $RA_3 = 2$ (2 étant la capacité consommée dans l'intervalle $[8, 11]$).

Algorithmes à vol de temps creux (Slack-stealing algorithms) - (Lehoczky et Ramos-Thuel)

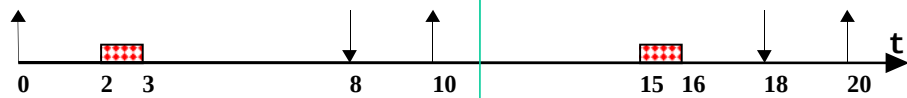
- Pour les systèmes dirigés par les délais (ex. EDF ou LLF)
- Aucune tâche périodique n'est dédiée au traitement des tâches apériodiques.
- Principe de fonctionnement :
lorsqu'une tâche apériodique est activée, le système recule au maximum l'instant de déclenchement des tâches périodiques pour autant qu'elles continuent à respecter leurs contraintes temporelles.
Quand EDF est utilisé, on calcule pour chaque tâche apériodique une échéance fictive **df** qui est celle qu'il faut lui associer pour qu'elle soit exécutée avec un temps de réponse minimal. **df** correspond alors à la première date pour laquelle le temps d'exécution de la tâche est égal au temps total d'inactivité du processeur quand l'ordonnancement respecte les échéances de toutes les tâches périodiques et des tâches apériodiques précédemment déclenchées et non encore achevées.

Slack-stealing avec EDF : exemple (1/2)

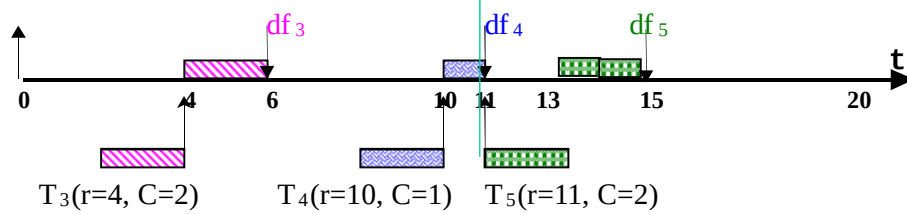
$T_1(r_0=0, C=2, D=4, P=5)$



$T_2(r_0=0, C=1, D=8, P=10)$



Tâches apériodiques



Slack-stealing : exemple (2/2)

- À $t=4$, la tâche apériodique $T_3(r=4, C=2)$ est activée.
- La deuxième requête de la tâche T_1 se réveillant à $t=5$ a une laxité de 2 unités de temps ; elle peut donc reculer le début de son exécution jusqu'à $t=6$.
- La tâche T_3 peut donc s'exécuter immédiatement entre $t=4$ et $t=6$ et sa date d'échéance fictive df_3 est alors égale à 6.
- De la même façon, la troisième requête de la tâche T_1 peut reculer le début de son exécution jusqu'à $t=11$ de façon à laisser la tâche T_4 s'exécuter immédiatement suite à leur réveil. La tâche T_4 a donc une échéance fictive fixée à $df_4=11$.
- La tâche T_5 obtient une échéance fictive $df_5=15$ et s'exécute après la fin de la troisième requête de T_1 (c'est-à-dire dans l'intervalle $[13,15]$).

Comparaison des algorithmes de traitement de tâches apériodiques

Technique de traitement	Performance (opt. du temps de rép.)	Complexité de calcul	Complexité implémentation
Traitement en arrière plan	↘	↗	↗
Serveur à scrutation	↘	↗	↗
Serveur ajournable	→	↗	↗
Serveur sporadique	→	→	→
Slack-Stealer	↗	↘	↘

↗ : excellent → : bon ↘ : mauvais

Théorèmes de Tia, Liu et Shankar (1995)

- 1.** Il n'existe pas d'algorithme valide qui minimise le temps de réponse pour chaque tâche apériodique à contraintes relatives, pour tout ensemble de tâches périodiques ordonnées par priorités fixes et tout ensemble de tâches apériodiques ordonnées selon une discipline donnée.

- 2.** Il n'existe pas d'algorithme (en ligne) valide qui minimise le temps de réponse moyen des tâches apériodiques à contraintes relatives, pour tout ensemble de tâches périodiques ordonnées par priorités fixes et tout ensemble de tâches apériodiques ordonnées selon une discipline donnée.